Principios y Prácticas de Desarrollo de Software Orientado a Objetos

Comportamiento de una aplicación bajo diferentes escenarios de despliegue

Docker Compose vs. Kubernetes (K3s) con 1, 2 y 3 nodos · Proyecto Final

David Esteban Suarez Lozano · 2221909
Thomas Alejandro Perez Rojas · 2221906
Jesus David Santodomingo Carrascal · 2221922

FastAPI PostgreSQL Docker K3s JMeter Prometheus

Agenda

1. **Contexto**
   Qué pide el proyecto y la matriz de escenarios
2. **Aplicación**
   La tienda online y sus endpoints
3. **Infraestructura**
   Google Cloud, Docker Compose y K3s
4. **Metodología**
   JMeter, 120 corridas y monitoreo
5. **Resultados**
   Throughput, escalado y cuello de botella
6. **Hallazgos y cierre**
   Sondas, /dev/shm, conclusiones

1 · Contexto

Medir qué cambia al orquestar y escalar una aplicación

Lo que pide el proyecto y cómo lo cumplimos

Requisito	Cómo lo resolvimos
Aplicación con BD de ≥ 10.000 registros	Tienda FastAPI + PostgreSQL con ~920.000 registros
Despliegue con Docker	Docker Compose en nodo1 (línea base)
Kubernetes con 1, 2 y 3 nodos	Clúster K3s en Google Cloud, un nodo más por fase
Distinto número de réplicas	1, 2 y 3 réplicas de la API en cada fase
Pruebas de carga desde otra máquina	JMeter 5.6.3 en vm-jmeter, misma red privada
Monitorización (opcional)	Prometheus + Grafana + cAdvisor + node-exporter
Análisis en notebook	Jupyter / Colab con 8 gráficas y conclusiones

Matriz del experimento

10

escenarios

Docker + 3 nodos × 3 réplicas

120

corridas

4 cargas × 3 repeticiones

412 mil

peticiones

enviadas por JMeter

Escenario	1 réplica	2 réplicas	3 réplicas	Cargas (usuarios)
Docker Compose (1 VM)	●			10 · 50 · 100 · 200
Kubernetes · 1 nodo	●	●	●	10 · 50 · 100 · 200
Kubernetes · 2 nodos	●	●	●	10 · 50 · 100 · 200
Kubernetes · 3 nodos	●	●	●	10 · 50 · 100 · 200

2 · La aplicación

Una tienda online con lecturas, búsquedas, escrituras y reportes

Tienda online: FastAPI + PostgreSQL

Endpoint	Tipo de carga	Peso
GET /productos/{id}	Lectura por clave primaria	50 %
GET /productos?categoria=X	Lectura paginada con filtro	25 %
GET /productos/buscar?q=	ILIKE + conteo (toda la tabla)	10 %
POST /pedidos	Escritura transaccional	10 %
GET /reportes/ventas-por-categoria	JOIN + GROUP BY sobre 600 mil items	5 %

100.000 productos

20.000 clientes

200.000 pedidos

599.346 items de pedido

Datos generados con Faker y semilla fija: idénticos en todos los escenarios, y la BD se resetea antes de cada uno

Decisiones para una comparación justa

1 1 worker por contenedor

La unidad de escalado es la réplica, no el proceso

2 Mismos límites en todo

API: 1 CPU / 512 MB · PostgreSQL: 1 CPU / 2 GB

3 Pool acotado

10 + 5 conexiones por réplica: máximo 45 con 3

4 Una sola PostgreSQL

Fija en nodo1: solo se escala la API, como pide la guía

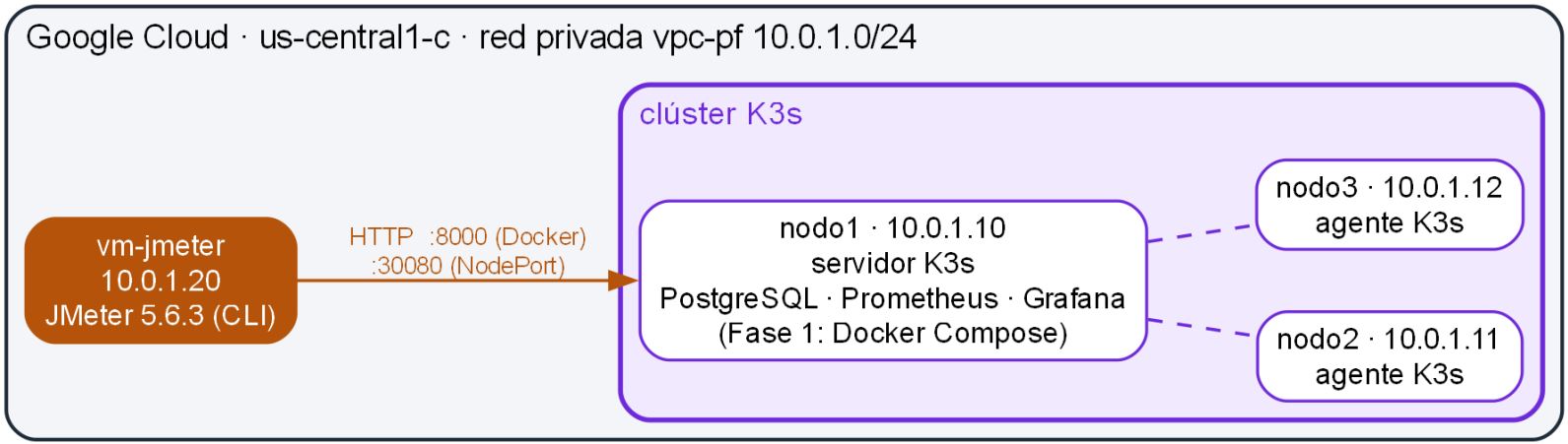
app/main.py · endpoint costoso (resumido)

```
@app.get("/productos/buscar")
async def buscar(q: str, ...):
  p = f"%{q}%"
  filtro = (nombre.ilike(p)
            | descripcion.ilike(p))
  # cuenta TODA la tabla:
  total = await s.scalar(
    select(count()).where(filtro))
  filas = await s.scalars(
    select(Producto).where(filtro)
    .limit(20))
  return {"total": total, ...}
```

3 · Infraestructura

Google Cloud, Docker Compose y Kubernetes (K3s)

4 VMs en la misma red privada



n2d-standard-2

2 vCPU AMD · 8 GB · no burstable (como D2as_v5)

JMeter aparte

Otra VM de la misma red: latencia baja y estable

De Azure a Google Cloud

Azure for Students no tenía cuota (6 vCPU, DASv5 = 0)

Fase 1: Docker Compose

docker-compose.yml (resumido)

```
services:
  db:
    image: postgres:16
    cpus: "1.0"
    mem_limit: 2g
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d tienda"]
  api:
    image: ghcr.io/tapr16/pf-api:1.0
    environment:
      DATABASE_URL: postgresql+asyncpg://app:...@db:5432/tienda
      DB_POOL_SIZE: "10"
    ports: ["8000:8000"]
    depends_on: { db: { condition: service_healthy } }
    cpus: "1.0"
    mem_limit: 512m
```

- Imagen publicada en GHCR por un GitHub Action: la misma en todas las fases
- Monitoreo aparte: Prometheus, Grafana, cAdvisor y node-exporter
- **Docker 29 + cAdvisor: hubo que volver al almacenamiento overlay2 para ver los contenedores**

Fases 2-4: Kubernetes con K3s

k8s/04-api-deployment.yaml (resumido)

```
kind: Deployment
metadata: { name: api, namespace: proyecto }
spec:
  replicas: 1          # se escala a 2 y 3
  template:
    spec:
      topologySpreadConstraints:
        - { maxSkew: 1, topologyKey: kubernetes.io/hostname,
            whenUnsatisfiable: ScheduleAnyway }
      containers:
        - name: api
          image: ghcr.io/tapr16/pf-api:1.0
          resources:
            requests: { cpu: 250m, memory: 512Mi }
            limits:   { cpu: "1",  memory: 512Mi }
          readinessProbe: { httpGet: { path: /health },
                            timeoutSeconds: 30 }
          livenessProbe:  { tcpSocket: { port: 8000 } }
```

1. **StatefulSet PostgreSQL**

   Fijo en nodo1 con nodeSelector rol=db

2. **NodePort 30080**

   Entrada única por nodo1; kube-proxy reparte

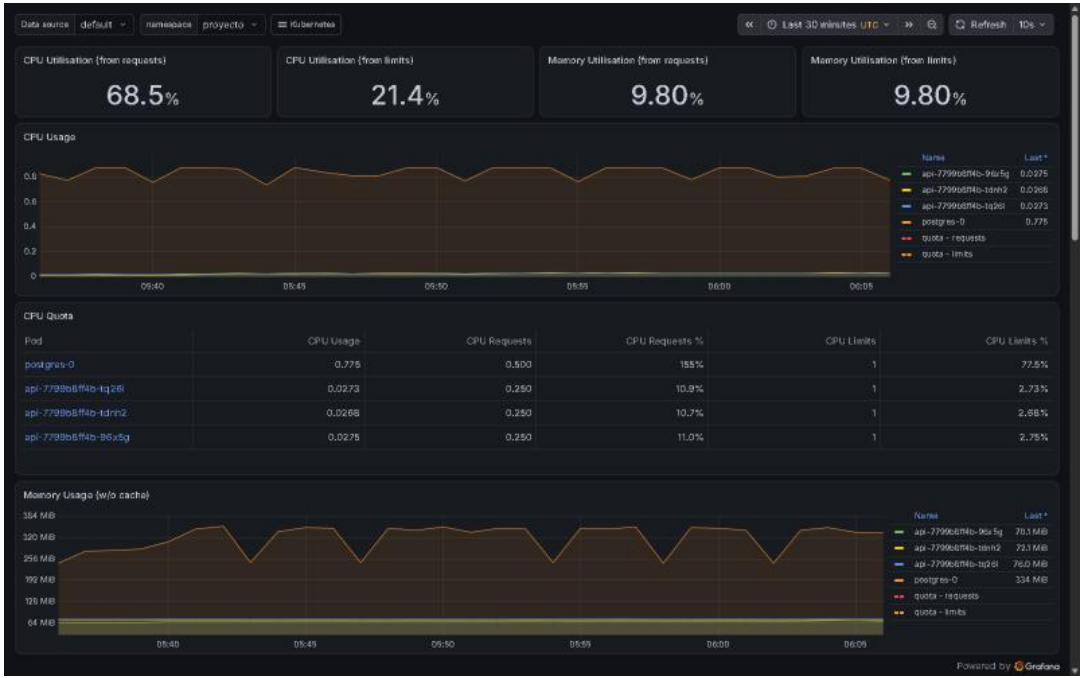
3. **Job seed**

   Resetea la BD antes de cada escenario

4. **Unión de nodos**

   k3s agent con el token de nodo1 + rollout restart

Monitorización con Prometheus y Grafana



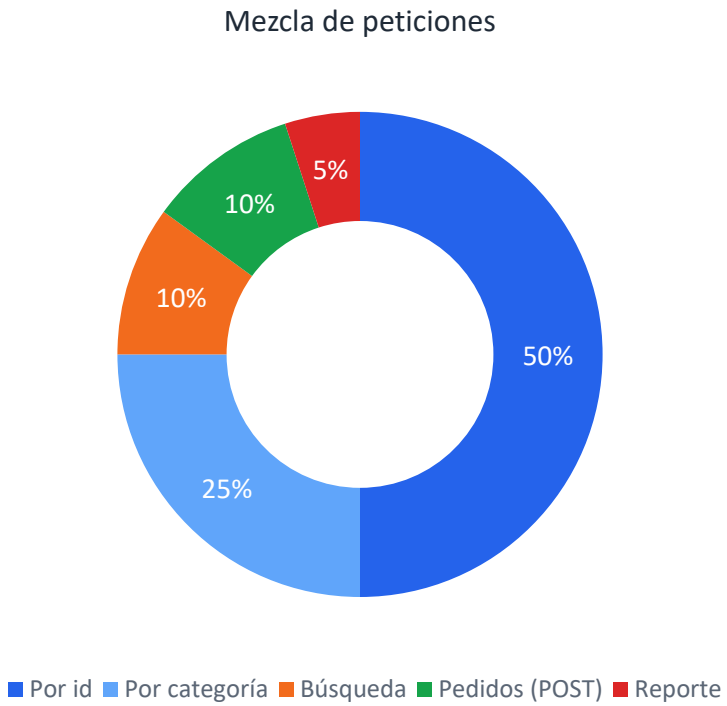
Grafana · 1 nodo, 3 réplicas, 200 usuarios: CPU por pod

- kube-prometheus-stack con Helm, fijo en nodo1
- node-exporter en cada nodo y ServiceMonitor para /metrics de la API
- Métricas exportadas a CSV por corrida para el notebook
- **Grafana en solo lectura y por túnel SSH: nada expuesto a internet**

4 · Metodología

Las mismas pruebas, con las mismas reglas, en cada escenario

Plan de carga con JMeter



Parámetro	Valor
Usuarios concurrentes	10 · 50 · 100 · 200
Duración por corrida	180 s (30 s de ramp-up)
Repeticiones	3 por nivel de carga
Calentamiento	60 s, descartado
Pausa entre corridas	30 s
Al analizar	Se quitan los primeros 30 s
Aserción	Código 200 o 201

Una fase de Kubernetes, automatizada

1

Resetear BD

Job seed: mismos datos

2

Escalar

1, 2 o 3 réplicas

3

Capturar

kubectl get pods -o wide

4

Medir

run_scenario.sh: 12 corridas

5

Exportar

Prometheus → CSV

vm-jmeter · jmeter/orquestar_fase.sh

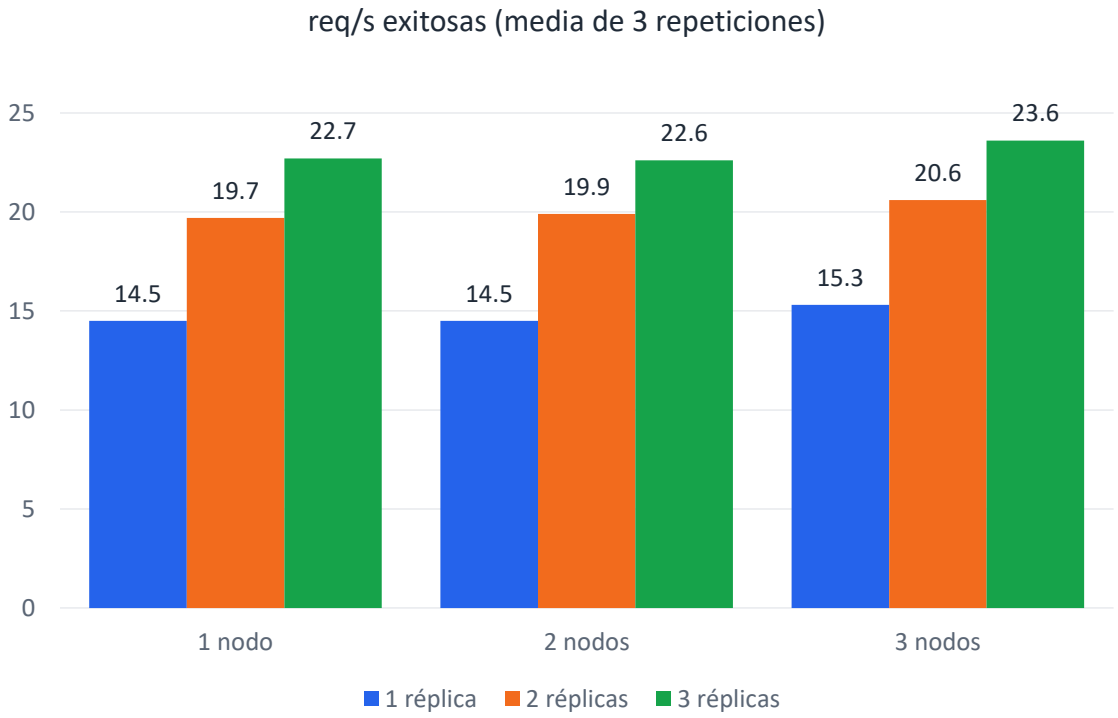
```
./orquestar_fase.sh 1      # k8s-1n-1r, k8s-1n-2r, k8s-1n-3r  (~2 h 20 min)
# unir nodo2 al clúster + kubectl rollout restart deployment/api
./orquestar_fase.sh 2      # k8s-2n-1r, k8s-2n-2r, k8s-2n-3r
./orquestar_fase.sh 3      # k8s-3n-1r, k8s-3n-2r, k8s-3n-3r
```

Validez: la CPU de vm-jmeter nunca pasó del 5 %, así que el generador de carga no limitó ninguna medición

5 · Resultados

120 corridas: qué mejora y qué no

Throughput con 200 usuarios: los 10 escenarios



Docker Compose

14,8 req/s

Mejor escenario

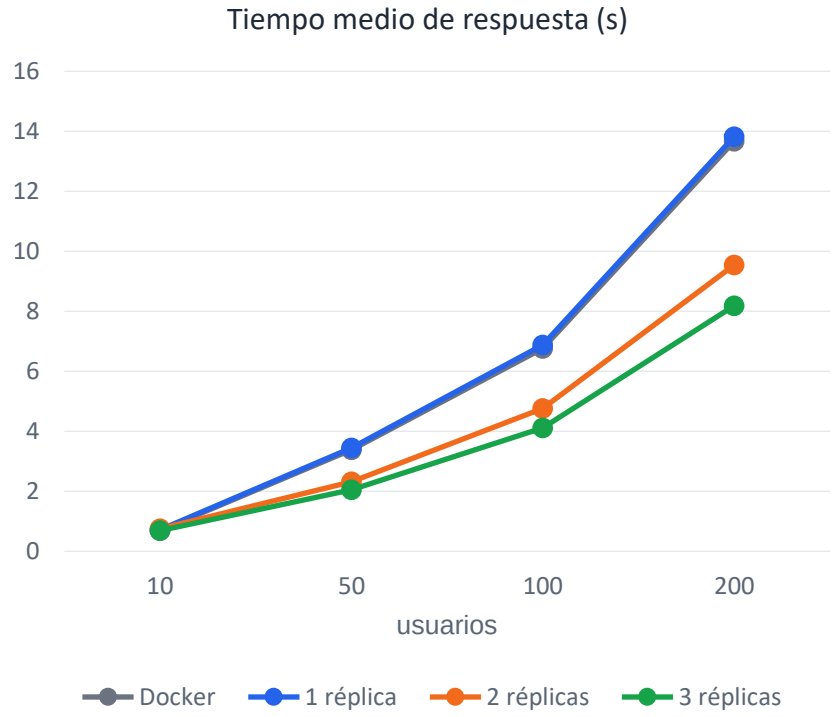
23,6 req/s

3 nodos · 3 réplicas · +59 % vs. Docker

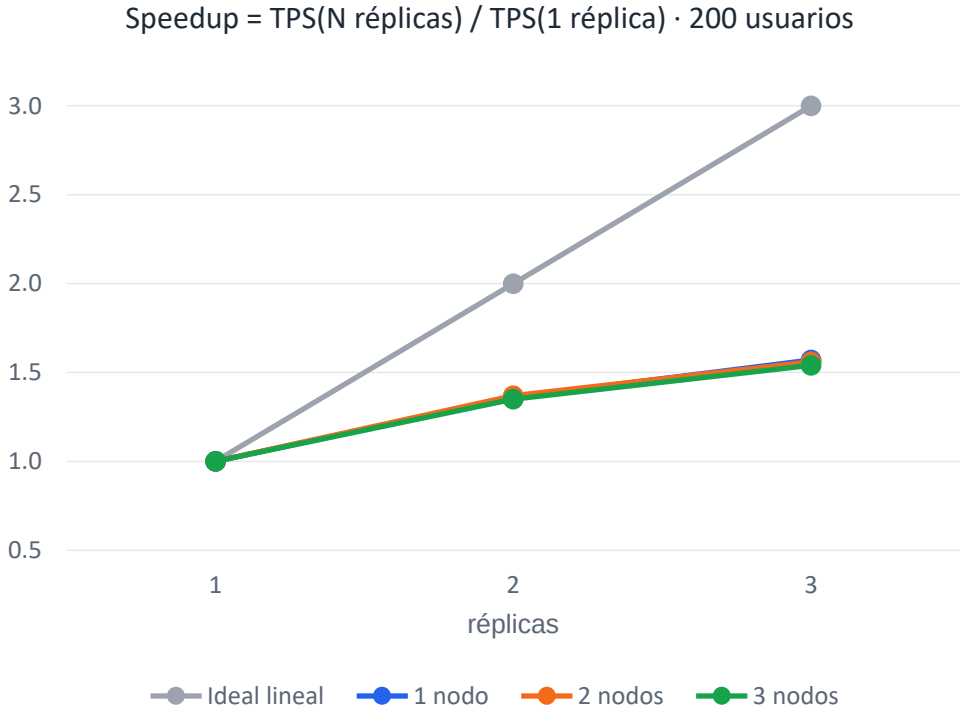
Las réplicas mueven el resultado; los nodos casi no

Saturación: el throughput se aplana pronto





Escalar réplicas: ~1,5× en vez de 3×



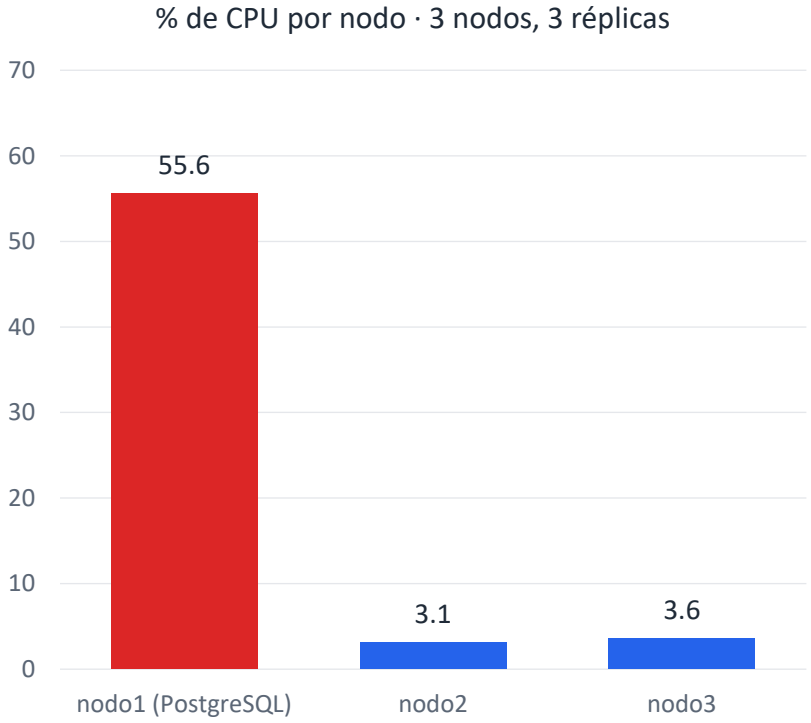
×1,35 / ×1,54

con 2 y 3 réplicas (ideal: ×2 y ×3)

¿Por qué sube algo?
Más réplicas = más conexiones simultáneas a PostgreSQL (15 → 30 → 45).

Pero con 2-3 réplicas falla el 70-85 % de los reportes: parte de la mejora es trabajo pesado que la BD deja de hacer.

Agregar nodos no ayuda

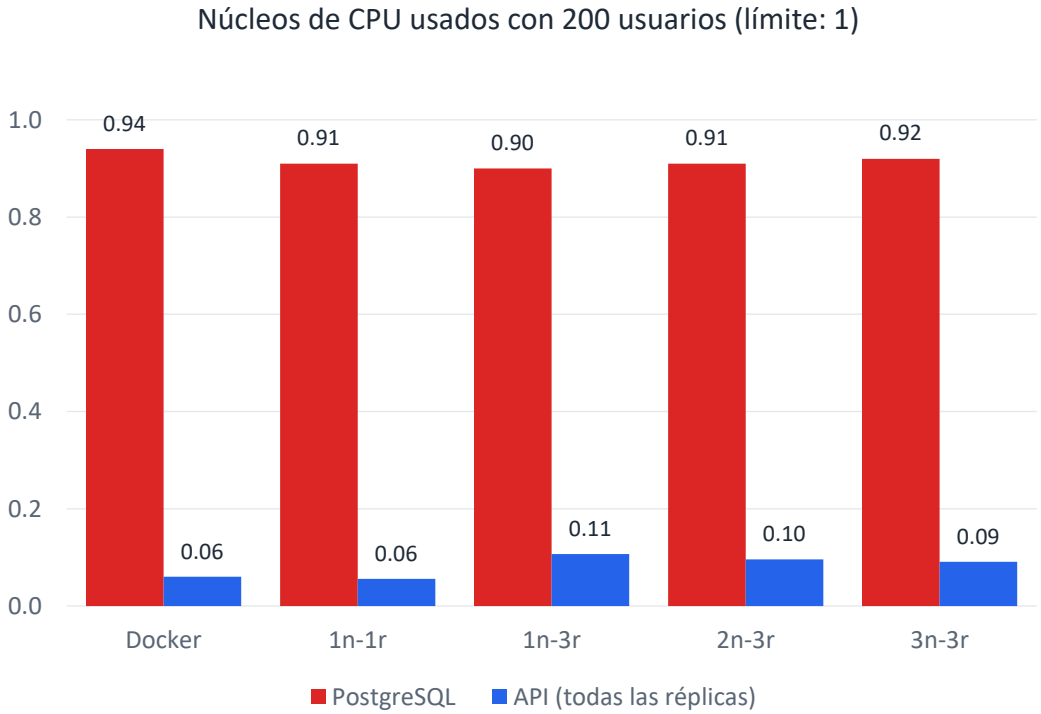




Prometheus · toda la fase de 3 réplicas

Con las mismas réplicas, pasar de 1 a 3 nodos cambia el throughput ≤ 5 %: la capacidad nueva queda ociosa

El cuello de botella es PostgreSQL



~0,9

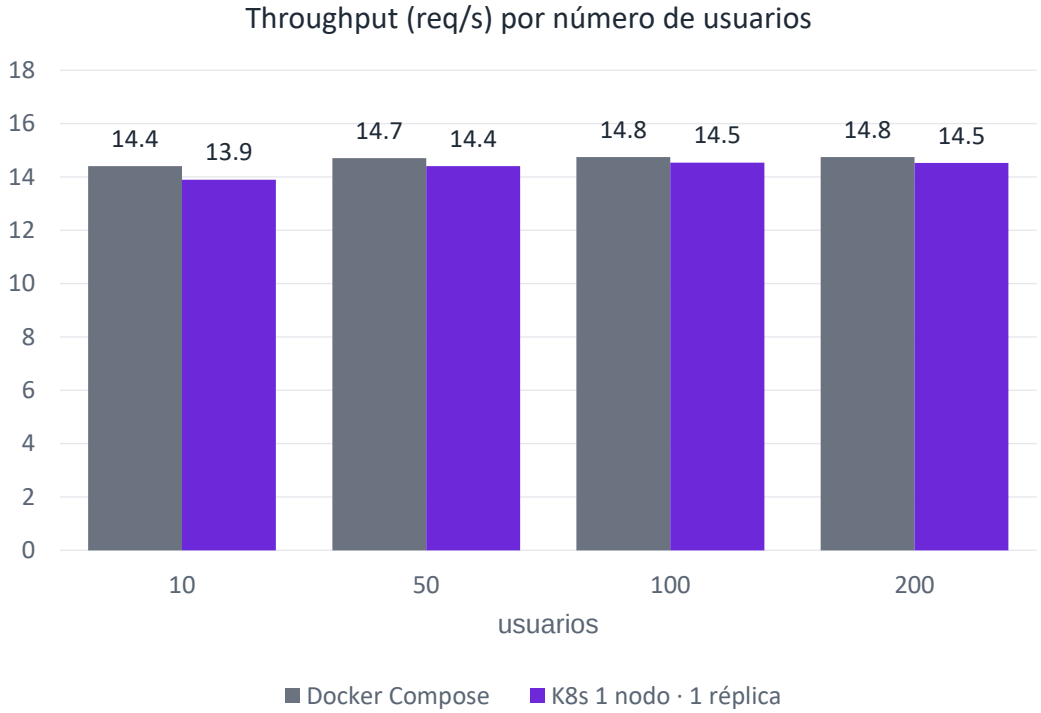
núcleos usa PostgreSQL en todos los escenarios

< 0,11

núcleos usan todas las réplicas de la API juntas

PostgreSQL además sufre throttling constante (~9 periodos limitados/s): choca con su límite todo el tiempo

Docker vs. Kubernetes: casi sin costo



–1,5 a –3,5 %

de throughput en Kubernetes respecto a Docker, con el mismo hardware

- Mismos tiempos de respuesta
- Errores < 1 % en ambos
- A cambio: escalado, autorreparación y reparto de carga

Demostración: escalado y carga en vivo

```
$ k -n proyecto scale deployment api --replicas=1 && k -n proyecto rollout status deployment/api
deployment.apps/api scaled
deployment "api" successfully rolled out

$ k -n proyecto get pods -l app=api -o wide
NAME                   READY   STATUS    RESTARTS   AGE    IP          NODE    NOMINATED NODE   READINESS GATES
api-6f4d7dbbdb-tcd4m   1/1     Running   0          114m   10.42.2.4   nodo3   <none>           <none>

$ k -n proyecto scale deployment api --replicas=3 && k -n proyecto rollout status deployment/api
deployment.apps/api scaled
Waiting for deployment "api" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "api" rollout to finish: 2 of 3 updated replicas are available...
deployment "api" successfully rolled out

$ k -n proyecto get pods -o wide
NAME                   READY   STATUS      RESTARTS       AGE    IP           NODE    NOMINATED NODE   READINESS G
api-6f4d7dbbdb-qrr8z   1/1     Running     0              20s    10.42.1.11   nodo2   <none>           <none>
api-6f4d7dbbdb-tcd4m   1/1     Running     0              114m   10.42.2.4    nodo3   <none>           <none>
api-6f4d7dbbdb-vdzq5   1/1     Running     0              20s    10.42.0.58   nodo1   <none>           <none>
```

kubectl scale --replicas=3: una réplica en cada nodo

```
  # Uso de CPU por nodo y por pod durante la carga (metrics-server):
$ k top nodes
NAME    CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
nodo1   1153m        57%      3179Mi          40%
nodo2   69m          3%       735Mi           9%
nodo3   64m          3%       502Mi           6%

$ k -n proyecto top pods
NAME                   CPU(cores)   MEMORY(bytes)
api-6f4d7dbbdb-qrr8z   38m          67Mi
api-6f4d7dbbdb-tcd4m   27m          72Mi
api-6f4d7dbbdb-vdzq5   19m          67Mi
postgres-0             999m         355Mi

  # PostgreSQL usa ~1 núcleo (su límite) y satura nodo1; nodo2 y nodo3 casi no trabajan:
  # agregar nodos no ayuda mientras la base de datos sea una sola instancia
```

kubectl top con 100 usuarios: postgres-0 usa 999m (1 núcleo)

Video del proyecto

10 min, sin audio, generado con OBS y una demo automática:

- Parte A · Docker Compose, API y JMeter
- Parte B · K3s con 3 nodos, escalado, kubectl top, Grafana y resultados

docs/video/Video-Proyecto-Final.mp4

6 · Hallazgos

Lo que aprendimos al medir de verdad

Las sondas por defecto tumbaban el servicio

Configuración (k8s-1n-1r)	50 us.	100 us.	200 us.
Sondas por defecto (timeout 1 s)	**19,2 %**	**87,2 %**	**68,9 %**
Readiness con 10 s	0,5 %	0,6 %	**6,0 %**
Sondas finales (usadas)	**0,5 %**	**0,4 %**	**0,9 %**

% de peticiones con error. Las corridas descartadas se guardan en results_descartados/

Corrección

```
readinessProbe:            # ¿recibe tráfico?
  httpGet: { path: /health }
  timeoutSeconds: 30
  failureThreshold: 8
livenessProbe:             # ¿está vivo?
  tcpSocket: { port: 8000 }
```

1

Qué pasaba

Bajo carga /health tardaba > 1 s (espera a la BD)

2

Efecto

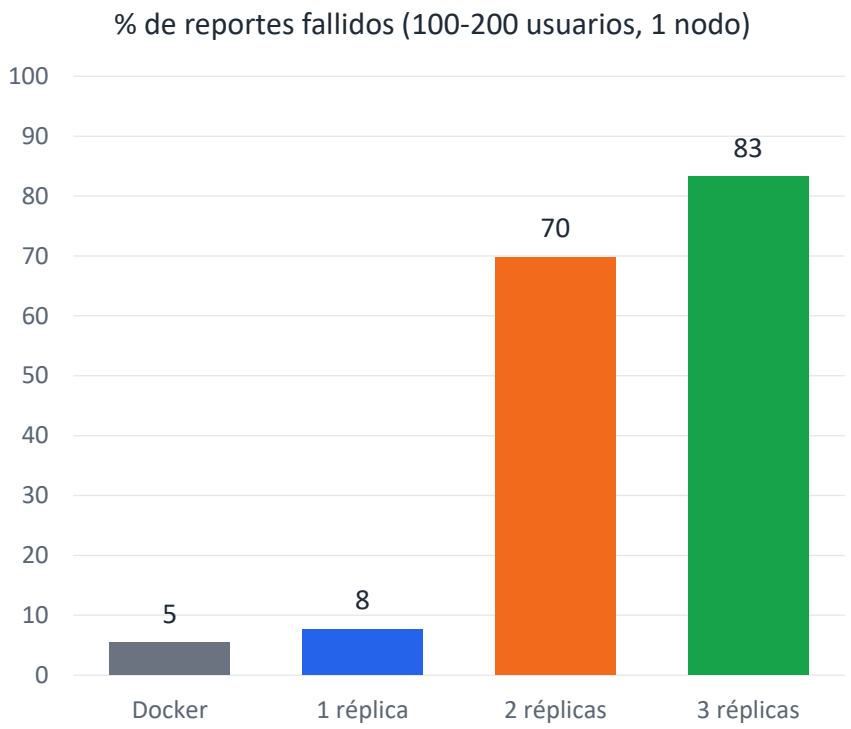
Kubernetes reiniciaba la API en plena prueba: CrashLoopBackOff, 9 reinicios

3

Lección

La sonda debe medir si el proceso vive, no si está rápido

Más réplicas agotan /dev/shm en PostgreSQL



log de PostgreSQL

```
ERROR: could not resize shared memory segment
       "/PostgreSQL.976963026" to 4194304 bytes:
       No space left on device
```

- El reporte usa parallel workers, que reservan memoria en /dev/shm (64 MB por defecto)
- 1 réplica: ≤ 15 conexiones, pocos reportes a la vez
- 3 réplicas: ≤ 45 conexiones y la memoria se agota → 500
- **Escalar la API traslada la presión a la base de datos**

Otros retos técnicos

1 **Cuota de Azure for Students**

6 vCPU y 0 para DASv5: migramos a Google Cloud con apagado automático de facturación

2 **cAdvisor no veía los contenedores**

Docker 29 usa el almacén de containerd: volvimos a overlay2

3 **Sin capacidad en la zona a**

Stockout de n2d en us-central1-a: script reintentable en la zona c

4 **IPs públicas que cambian**

Llaves SSH por nombre de VM y JMeter atacando la IP privada fija

Respuestas a las preguntas del proyecto

Pregunta	Respuesta
¿Crece el throughput con más réplicas?	Sí, pero no lineal: ×1,36 y ×1,55 (ideal ×2 y ×3)
¿Cuándo se satura?	1 réplica: desde 10 usuarios · 2-3 réplicas: desde 50
¿3 réplicas en 1 nodo = en 3 nodos?	Casi: 22,7 vs. 23,6 req/s (+4 %)
¿La BD es el cuello de botella?	Sí: ~0,9 de 1 núcleo con throttling; API < 0,11
¿Overhead de K8s vs. Docker?	Mínimo: entre −1,5 % y −3,5 %
¿Agregar nodos ayuda?	No con una sola BD: ≤ 5 % y nodos ociosos
¿Qué endpoint se degrada más?	Lecturas simples: de ~20 ms a ~12,7 s (×600)

Recomendaciones

1

Optimizar la BD

Índice pg_trgm para la búsqueda; vista materializada o Redis para el reporte

2

Escalar la BD

Más CPU o réplicas de lectura; PgBouncer para controlar las conexiones

3

/dev/shm más grande

emptyDir con medium: Memory (K8s) o shm_size (Compose)

4

Sondas tolerantes

Liveness por TCP y readiness con timeouts de la latencia real

5

HPA solo si la API limita

Con la API en < 0,11 núcleos no tiene sentido autoescalar

6

Ingress / balanceador

No depender del NodePort de un solo nodo

Entregables

1	**Repositorio**	**github.com/Tapr16/Proyecto-Final-Principios**	Código, manifiestos, scripts, resultados y README
2	**Notebook**	**notebook/analisis.ipynb (abre en Colab)**	Carga de las 120 corridas, 8 gráficas y análisis
3	**Video**	**docs/video/Video-Proyecto-Final.mp4**	10 min: Docker y Kubernetes funcionando en vivo
4	**Informe**	**PDF con instrucciones de ejecución**	Requisitos, pasos y cómo ver el notebook

Conclusiones

- El cuello de botella es la base de datos, no la aplicación: ni Docker ni Kubernetes lo cambian
- Kubernetes casi no añade costo sobre el mismo hardware (−1,5 % a −3,5 %)
- Escalar una capa sin estado ayuda poco si el estado está centralizado: ×1,55 con 3 réplicas
- Agregar nodos no sirve si lo que limita no se replica: dos nodos quedaron < 7 % de CPU
- Los valores por defecto de K8s pueden empeorar un sistema saturado: sondas de 1 segundo
- Sin métricas internas solo veíamos que no escalaba; Prometheus mostró por qué